

P1696R0

Refinement proposal for P0920 Precalculated hash values in lookup

Draft Proposal, 2019-06-12

This version:

<http://wg21.link/p1696r0>

Authors:

[Xiao Shi \(Facebook\)](#) xshi.fb@gmail.com

[Mateusz Pusz \(Epam Systems\)](#) mateusz.pusz@gmail.com

[Geoffrey Romer \(Google\)](#) gromer@google.com

[Jay Feldblum \(Facebook\)](#) yfeldblum@gmail.com

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

[\[P0920R2\]](#) proposed an extension of the lookup API of unordered associative containers that allows the user to precalculate hash values and use them directly. We have found the proposed API error-prone from implementation experiences. This paper details the problems and proposes corresponding design alternatives for using precalculated hash values.

Table of Contents

1 Motivation and Scope

- 1.1 Hasher misuse
- 1.2 Unclear semantics of sharing hash values across containers
- 1.3 Use of raw hash values limits optimization opportunities
- 1.4 Extensibility considerations

2 Design alternatives and respective concerns

- 2.1 Encapsulate hash values with `hash_token`
 - 2.1.1 Possible mismatch between `hash_token` and key
 - 2.1.2 Whether to store reference / copy of key in `hash_token`
 - 2.1.3 Sharing `hash_token`s across different containers
- 2.2 “Caching” hash values in keys

3 Proposed Wording

3.1 Option 1: Revert [P0920R2]

3.2 Option 2: Replace `size_t` with an opaque token

4 Implementation and Usage Experiences

5 Acknowledgements

References

Normative References

Informative References

§ 1. Motivation and Scope

[P0920R2] extends the interface of unordered containers with member function overloads that have one additional argument taking a precalculated hash value for the value being queried.

EXAMPLE 1

The motivating example below (taken from [P0920R2]) allows the code to look for the same keyword in more than one container at a time.

```
std::array<std::unordered_map<std::string, int>, array_size> maps;

void update(const std::string& user)
{
    const auto hash = maps.front().hash_function()(user);
    for(auto& m : maps) {
        auto it = m.find(user, hash);
        // ...
    }
}
```

This approach has a number of problems, including the lack of safety around hasher misuse and the lack of clarity around sharing hash values across multiple containers.

§ 1.1. Hasher misuse

Instead of obtaining the hasher instance via `map.hash_function()`, users often precalculate the key using a different hasher type or the wrong hasher instance. (See the [§4 Implementation and Usage Experiences](#) section for more details.)

Firstly, this is plain incorrect if the hasher for the container is stateful. Secondly, even if the user used a different hasher instance of the correct hasher type and the hasher is not stateful, it increases the chance of mismatch between the hash value and the key, especially when the hasher of the container gets changed. It would require auditing and potentially changing all call sites of this API.

§ 1.2. Unclear semantics of sharing hash values across containers

The motivating example in [§1 Motivation and Scope](#) assumes the hasher type `std::hash<std::string>` is not stateful and the maps in the array all use the same hasher type. This assumption may not be true for other use cases or implementations, and is susceptible to future changes. For example, if the different maps are in different part of a code base, there is no way for the user to tell whether hash values from one `unordered_map` can be reused in another other than comparing the types of the hashers.

§ 1.3. Use of raw hash values limits optimization opportunities

Some hash table implementations may choose to further process the hash values obtained from `Hash`. For example, [\[F14\]](#) mixes hash values from hashers not known to be avalanching (e.g., `std::hash` for numeric types). The existing approach does not allow hash table implementations to preprocess.

In addition, in a few use cases, sizable performance gains came mainly from prefetching the pertinent cache lines, as opposed to merely precalculating the hash value. This optimization is not possible with the API [\[P0920R2\]](#) proposed.

§ 1.4. Extensibility considerations

In the future, we may wish to extend the mutation methods (`emplace`, `try_emplace`, `insert`, etc.) with both heterogeneous operations and precalculated hash API. Keeping the position of the precalculated hash argument consistent and putting it as the first argument would be advantageous.

EXAMPLE 2

If the `mapped_type` is constructible both from an N-tuple that starts with `size_t` as well as an (N-1)-tuple, overload resolution will be unable to resolve the following:

```
try_emplace(const key_type& k, size_t hash, Args&&... args);
try_emplace(const key_type& k, Args&&... args);
```

For `emplace`, disambiguation is more difficult, which suggests an API akin to `emplace_hint` where the *first* parameter is the precalculated hash / position hint.

§ 2. Design alternatives and respective concerns

§ 2.1. Encapsulate hash values with `hash_token`

Some of the above issues can be addressed by encapsulating the raw hash value in a `hash_token` and introducing a API which allows the container to return an opaque token to the user that can be used in lookups.

EXAMPLE 3

```
std::unordered_map<std::string, int> m;  
auto ht = m.token(key);  
m.find(ht, key);
```

Since the token can only be meaningfully generated via the member function `token()`, it prevents misuse of the hasher instance. However, it does *not* preclude possible mismatch between `hash_token` and key.

§ 2.1.1. Possible mismatch between `hash_token` and key

The intention of lookup using precalculated hash is to improve performance in specific use cases which need it. These use cases can presumably be trusted to provide a matching `hash_token` and key. The semantics of the look up API would be "find the table entry identified by the key if the provided `hash_token` matches the key, returning an iterator to the entry, otherwise returning `end()`."

Conceivably, the hash table implementation could check whether the supplied `hash_token`s match the keys in debug builds. (Note that we could do that with raw hash values as well and we acknowledge this is an unresolved problem with this approach.)

§ 2.1.2. Whether to store reference / copy of key in `hash_token`

Storing a reference or a copy of the key in the `hash_token` would prevent any mismatch between the `hash_token` and the key, but could easily result in dangling references or expensive key copies.

§ 2.1.3. Sharing `hash_token`s across different containers

This approach does not provide a satisfying solution to specifying sharing semantics across different containers.

§ 2.2. "Caching" hash values in keys

Another approach for using precalculating hash is to "cache" hash values alongside `key_types` and rely on heterogeneous lookup ([P0919R3], P1690R0) to use the precalculated hash values. (P1661R0 has a detailed account of a similar approach.)

EXAMPLE 4

```
struct KeyWithHash {
    std::string key_;
    size_t hash_;
};

std::unordered_map<std::string, int, MyHash, MyEqualTo> m;
m.emplace("foo", 42);

KeyWithHash kh;
kh.key_ = "foo";
kh.hash_ = m.hash_function()(kh.key_); // precalculated
m.find(kh); // heterogeneous lookup
```

This approach allows the user to exploit the benefit of precalculating hash without standard library API changes. It does, however, put more onus on the user, who is required to implement a custom hasher and key equality comparison operator. It does *not* address some of the problems stated above, as the user is still required to use the right hasher instance and the raw hash value is still exposed.

Note that this approach does not need the committee need to adopt any changes to the standard. This is an option available to users with or without [\[P0920R2\]](#). This could also be a mitigation that would buy us time to explore the design space further.

§ 3. Proposed Wording

§ 3.1. Option 1: Revert [\[P0920R2\]](#)

Given the issues detailed above, as well as the lack of an obviously clean design or agreeable approach to use precalculated hash values for lookups, we recommend reverting the changes in [\[P0920R2\]](#), as the first option. This allows the LEWG to come to a safer and better-explored design in the near future.

§ 3.2. Option 2: Replace `size_t` with an opaque token

As a second option, we recommend using an opaque hash token to encapsulate the generation and usage of precalculated hash, with the following wording changes. This is a relatively small forward-fix to [\[P0920R2\]](#).

The proposed changes are relative to the working draft of the standard as of [\[N4810\]](#).

Modify **22.2.7 [unord.req]** paragraph 11.23 as follows:

`hk` and `hke` denote values of type `size_t X::hash_token_type`.

Add the following paragraph to **22.2.7 [unord.req]**:

Member function templates `find`, `count`, `equal_range`, and `contains` that take a `hash_token_type` shall not participate in overload resolution unless `std::is_empty_v<Hash>` is true.

Modify Table 70 in **22.2.7 [unord.req]** as follows:

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>X::hash_token_type</code>	An implementation-defined type that can be used to look up its associated key.	<i>Expects:</i> <code>hash_token_type</code> meets the <code>Cpp17DefaultConstructible</code> requirements.	compile time
<code>b.token(k)</code>	<code>X::hash_token_type</code>	<i>Returns:</i> returns a value <code>hk</code> of type <code>X::hash_token_type</code> such that <code>b.find(hk, k)</code> equals <code>b.find(k)</code> .	constant
<code>a_tran.token(ke)</code>	<code>X::hash_token_type</code>	<i>Returns:</i> returns a value <code>hke</code> of type <code>X::hash_token_type</code> such that <code>a_tran.find(hke, ke)</code> equals <code>a_tran.find(ke)</code> .	constant
<code>b.find(k, hk)</code> <code>b.find(hk, k)</code>	iterator; <code>const_iterator</code> for const <code>b</code> .	<i>Expects:</i> <code>b.hash_function()(k)</code> equals <code>hk</code>; <code>b.token(k)</code> equals <code>hk</code> , <i>Returns:</i> an iterator pointing to an element with key equivalent to <code>k</code> , or <code>b.end()</code> if no such element exists.	Average case $O(1)$, worst case $O(b.size())$.
<code>a_tran.find(ke, hke)</code> <code>a_tran.find(hke, ke)</code>	iterator; <code>const_iterator</code> for const <code>a_tran</code> .	<i>Expects:</i> <code>a_tran.hash_function()(ke)</code> equals <code>hke</code>; <code>a_tran.token(ke)</code> equals <code>hke</code> , <i>Returns:</i> an iterator pointing to an element with key equivalent to <code>ke</code> , or <code>a_tran.end()</code> if no such element exists.	Average case $O(1)$, worst case $O(a_tran.size())$.
<code>b.count(k, hk)</code> <code>b.count(hk, k)</code>	<code>size_type</code>	<i>Expects:</i> <code>b.hash_function()(k)</code> equals <code>hk</code>; <code>b.token(k)</code> equals <code>hk</code> , <i>Returns:</i> the number of elements with key equivalent to <code>k</code> .	Average case $O(b.count(k))$, worst case $O(b.size())$.
<code>a_tran.count(ke, hke)</code> <code>a_tran.count(hke, ke)</code>	<code>size_type</code>	<i>Expects:</i> <code>a_tran.hash_function()(ke)</code> equals <code>hke</code>; <code>a_tran.token(ke)</code> equals <code>hke</code> , <i>Returns:</i> the number of	Average case $O(a_tran.count(ke))$, worst case $O(a_tran.size())$.

		elements with key equivalent to <code>ke</code> .	
<code>b.contains(k, hk)</code> <code>b.contains(hk, k)</code>	bool	<i>Expects:</i> <code>b.hash_function()(k)</code> <code>equals hk;</code> <code>b.token(k)</code> <code>equals hk,</code> <i>Effects:</i> Equivalent to <code>b.find(k, hk) !=</code> <code>b.end()</code>	Average case $O(1)$, worst case $O(b.size())$
<code>a_tran.contains(ke, hke)</code> <code>a_tran.contains(hke, ke)</code>	bool	<i>Expects:</i> <code>a_tran.hash_function()(ke)</code> <code>(ke) equals hke;</code> <code>a_tran.token(ke) equals</code> <code>hke,</code> <i>Effects:</i> Equivalent to <code>a_tran.find(ke, hke) !=</code> <code>a_tran.end()</code>	Average case $O(1)$, worst case $O(a_tran.size())$
<code>b.equal_range(k, hk)</code> <code>b.equal_range(hk, k)</code>	<code>pair<iterator, iterator>;</code> <code>pair<const_iterator, const_iterator></code> for const <code>b</code> .	<i>Expects:</i> <code>b.hash_function()(k)</code> <code>equals hk;</code> <code>b.token(k)</code> <code>equals hk,</code> <i>Returns:</i> a range containing all elements with keys equivalent to <code>k</code> . Returns <code>make_pair(b.end(),</code> <code>b.end())</code> if no such elements exist.	Average case $O(b.count(k))$, worst case $O(b.size())$.
<code>a_tran.equal_range(ke, hke)</code> <code>a_tran.equal_range(hke, ke)</code>	<code>pair<iterator, iterator>;</code> <code>pair<const_iterator, const_iterator></code> for const <code>a_tran</code> .	<i>Expects:</i> <code>a_tran.hash_function()(ke)</code> <code>(ke) equals hke;</code> <code>a_tran.token(ke) equals</code> <code>hke,</code> <i>Returns:</i> a range containing all elements with keys equivalent to <code>ke</code> . Returns <code>make_pair(a_tran.end(),</code> <code>a_tran.end())</code> if no such elements exist.	Average case $O(a_tran.count(ke))$, worst case $O(a_tran.size())$.

Make the following changes to:

- 22.5.4.1 [unord.map.overview]
- 22.5.5.1 [unord.multimap.overview]
- 22.5.6.1 [unord.set.overview]
- 22.5.7.1 [unord.multiset.overview]


```

using const_local_iterator = implementation-defined; // see 22.2
using node_type            = unspecified;
using insert_return_type   = insert-return-type<iterator, node_type>;
using hash_token_type      = implementation-defined; // see 22.2
...
hash_token_type            token(const key_type& k);
template <class K> hash_token_type token(const K& k);
iterator                   find(const key_type& k);
const_iterator             find(const key_type& k) const;
iterator                   find(const hash_token_type& hash_token, const key_type& k, size_t hash);
const_iterator             find(const hash_token_type& hash_token, const key_type& k, size_t hash) const;
template <class K> iterator find(const K& k);
template <class K> const_iterator find(const K& k) const;
template <class K> iterator find(const hash_token_type& hash_token, const K& k, size_t hash);
template <class K> const_iterator find(const hash_token_type& hash_token, const K& k, size_t hash) const;
size_type count(const key_type& k) const;
size_type count(const hash_token_type& hash_token, const key_type& k, size_t hash) const;
template <class K> size_type count(const K& k) const;
template <class K> size_type count(const hash_token_type& hash_token, const K& k, size_t hash) const;
bool contains(const key_type& k) const;
bool contains(const hash_token_type& hash_token, const key_type& k, size_t hash) const;
template <class K> bool contains(const K& k) const;
template <class K> bool contains(const hash_token_type& hash_token, const K& k, size_t hash) const;
pair<iterator, iterator>    equal_range(const key_type& k);
pair<const_iterator, const_iterator> equal_range(const key_type& k) const;
pair<iterator, iterator>    equal_range(const hash_token_type& hash_token, const key_type& k);
pair<const_iterator, const_iterator> equal_range(const hash_token_type& hash_token, const key_type& k) const;
template <class K> pair<iterator, iterator>    equal_range(const K& k);
template <class K> pair<const_iterator, const_iterator> equal_range(const K& k) const;
template <class K> pair<iterator, iterator>    equal_range(const hash_token_type& hash_token, const K& k);
template <class K> pair<const_iterator, const_iterator> equal_range(const hash_token_type& hash_token, const K& k) const;

```

§ 4. Implementation and Usage Experiences

Google’s widely used hash table implementation, [\[SwissTables\]](#), provides the exact same API as [\[P0929R0\]](#). There are fewer than a dozen or so uses of the precalculated hash in `find()`. None of them used the table’s hash instance (the correct one).

Facebook’s [\[F14\]](#) hash tables used an opaque hash token approach. In our experiences, such an approach allowed several use cases to precalculate hash and prefetch the relevant cache lines, resulting in CPU and memory efficiency improvement.

§ 5. Acknowledgements

We would like to thank Samuel Benzaquen, Nathan Bronson, David Goldblatt, Chris Kennelly, Matthew Fowles Kulukundis, and Titus Winters for various discussions, insights, and implementation experiences.

§ References

§ Normative References

[N4810]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4810). 15 March 2019. URL: <https://wg21.link/n4810>

[P0919R3]

Mateusz Pusz. [Heterogeneous lookup for unordered containers](https://wg21.link/p0919r3). 9 November 2018. URL: <https://wg21.link/p0919r3>

[P0920R2]

Mateusz Pusz. [Precalculated hash values in lookup](https://wg21.link/p0920r2). 22 February 2019. URL: <https://wg21.link/p0920r2>

[P0929R0]

Jens Maurer. [Checking for abstract class types](https://wg21.link/p0929r0). 8 February 2018. URL: <https://wg21.link/p0929r0>

§ Informative References

[F14]

Nathan Bronson; Xiao Shi. [Open-sourcing F14 for faster, more memory-efficient hash tables](https://code.fb.com/developer-tools/f14/). URL: <https://code.fb.com/developer-tools/f14/>

[SwissTables]

Sam Benzaquen; et al. [Swiss Tables and absl::Hash](https://abseil.io/blog/20180927-swisstables). URL: <https://abseil.io/blog/20180927-swisstables>